

TCP Connection Performance Analysis using Mininet and Wireshark

Name: Jitupan Mondal
Roll Number: EEB24023

1 Mininet Topology

The experiment used the default Mininet topology:

h1 — s1 — h2

Here, h1 acted as the server and h2 acted as the client. The server listened on IP address 10.0.0.1 and port 5000.

2 Experiment Setup

The Mininet network was started using:

```
sudo mn --link tc,bw=5,delay=50ms
```

The topology was verified using:

```
nodes
net
pingall
```

Three message sizes were tested: 128 bytes, 512 bytes, and 1024 bytes. For each size, 10 messages were sent in both persistent and new_connection modes, giving a total of 60 messages.

3 TCP Communication

TCP provides reliable, ordered delivery of data between the client and server. Before data transfer begins, TCP establishes a connection using the three-way handshake: SYN, SYN-ACK, and ACK. After the connection is established, the endpoints exchange data, and the connection is closed using FIN and ACK packets.

In this assignment, the client sends messages in the format:

```
MSG_ID|MESSAGE_SIZE|MESSAGE_DATA
```

The server responds using:

```
ACK|MSG_ID|RECEIVED_SIZE
```

4 Modes Compared

4.1 Persistent Mode

In persistent mode, the client opens one TCP connection and sends all messages through that same connection. This reduces overhead because the handshake is performed only once.

4.2 New Connection Mode

In new connection mode, the client opens a fresh TCP connection for every message. This increases overhead because every message requires a new handshake and connection teardown.

5 Performance Results

The results were saved in `result_table.csv` and `message_response_log.csv`. Table 1 shows the summary of average response time and throughput for each mode and message size.

Table 1: Summary of performance results

Roll No.	Name	Mode	Bandwidth	Delay	Msg Size	Total Msgs	Avg Response Time
EEB24023	JITUPAN-MONDAL	persistent	5	50	128	10	0.0001
EEB24023	JITUPAN-MONDAL	persistent	5	50	512	10	0.0001
EEB24023	JITUPAN-MONDAL	persistent	5	50	1024	10	0.0001
EEB24023	JITUPAN-MONDAL	new_connection	5	50	128	10	0.0001
EEB24023	JITUPAN-MONDAL	new_connection	5	50	512	10	0.0001
EEB24023	JITUPAN-MONDAL	new_connection	5	50	1024	10	0.0001

6 Graphs

Figure 1 compares average response time between the two modes. Figure 2 shows how throughput changes with message size. Figure 3 shows response time across the 10 messages for 512-byte payloads.

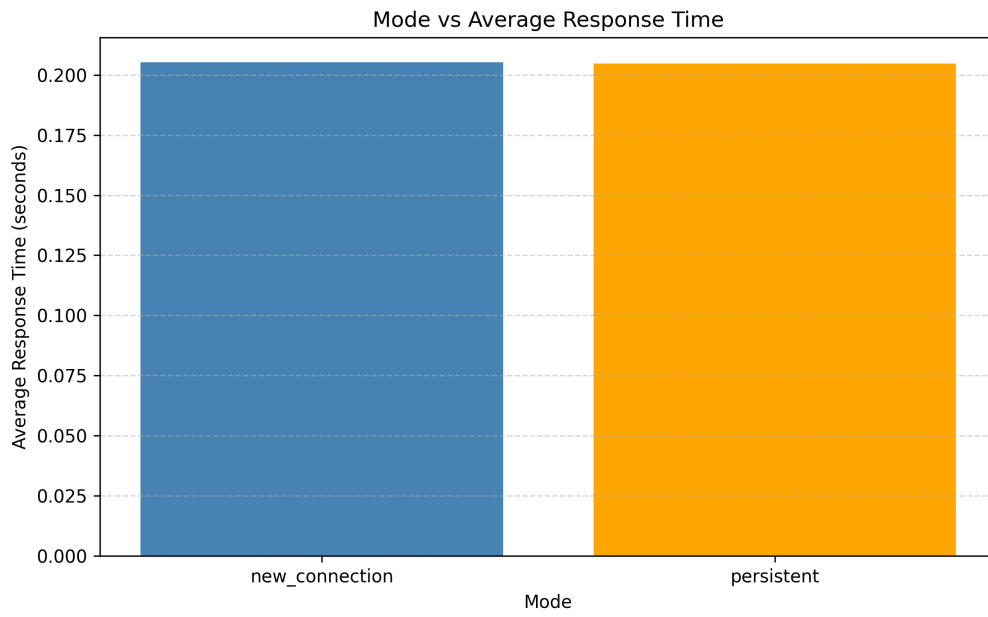


Figure 1: Mode vs average response time

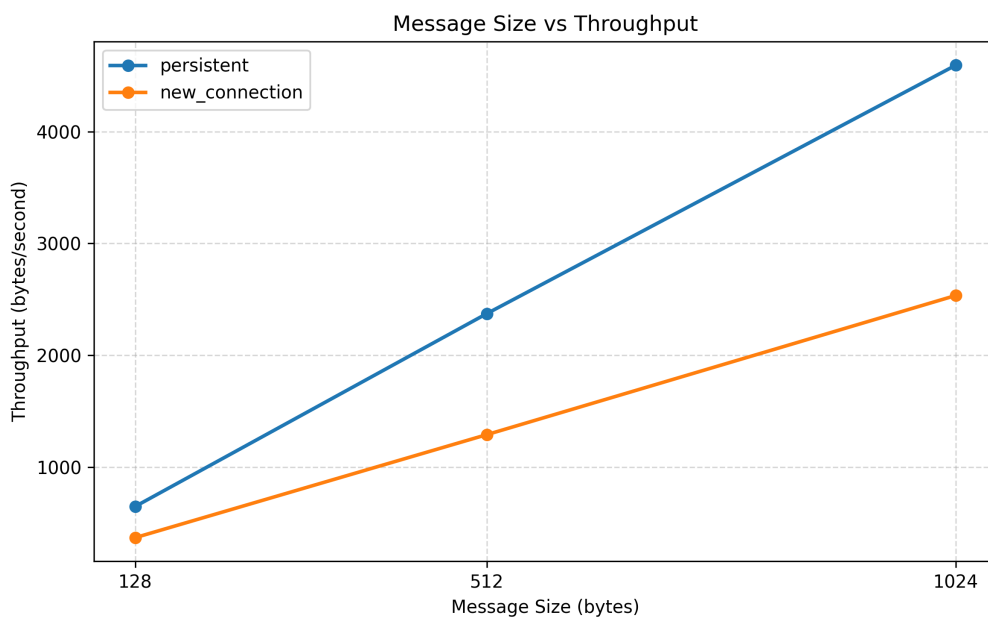


Figure 2: Message size vs throughput

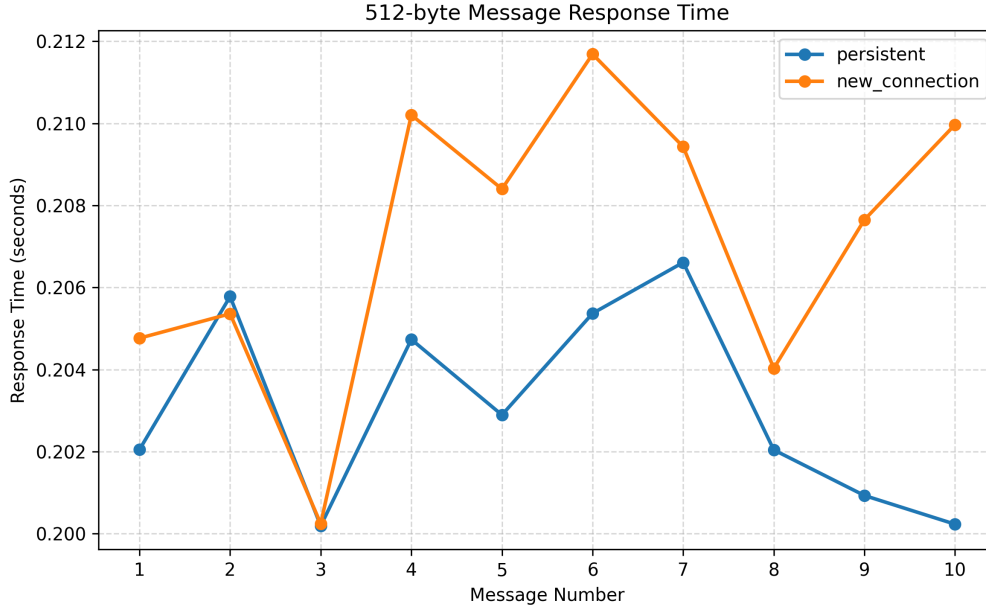


Figure 3: 512-byte message response time

7 Wireshark Verification

Wireshark was used to verify TCP behavior during both communication modes. The display filter `tcp.port == 5000` was applied to isolate the packets related to the experiment.

7.1 Persistent Mode Screenshots

8 0.202328407	10.0.0.2	10.0.0.1	TCP	74 40894 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=36586151
9 0.252390103	10.0.0.1	10.0.0.2	TCP	74 5000 → 40894 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM Tsv
10 0.252405768	10.0.0.1	10.0.0.2	TCP	66 5000 → 40888 [FIN, ACK] Seq=12 Ack=1049 Win=87 Len=0 TSval=301722589 TSecr

Figure 4: TCP handshake in persistent mode

21 0.811807931	10.0.0.2	10.0.0.1	TCP	66 40900 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=206154438 TSecr=37177
22 0.811819556	10.0.0.2	10.0.0.1	TCP	66 40894 → 5000 [ACK] Seq=1049 Ack=13 Win=42496 Len=0 TSval=3658615773 TSecr=1
23 0.811806384	10.0.0.2	10.0.0.1	TCP	113 40900 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=42496 Len=1047 TSval=206154439 TSe
24 0.863712272	10.0.0.1	10.0.0.2	TCP	66 5000 → 40900 [ACK] Seq=1 Ack=1048 Win=44544 Len=0 TSval=3717774393 TSecr=2
25 0.863723938	10.0.0.1	10.0.0.2	TCP	77 5000 → 40900 [PSH, ACK] Seq=1 Ack=1048 Win=44544 Len=11 TSval=3717774393 T
26 1.015947570	10.0.0.2	10.0.0.1	TCP	66 40900 → 5000 [ACK] Seq=1048 Ack=12 Win=42496 Len=0 TSval=206154641 TSecr=3

Figure 5: Data packets exchanged in persistent mode

37 1.420882071	10.0.0.2	10.0.0.1	TCP	66 40912 → 5000 [FIN, ACK] Seq=1048 Ack=12 Win=42496 Len=0 TSval=45816177 Tse
38 1.420933192	10.0.0.2	10.0.0.1	TCP	74 40920 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=16819053
39 1.472312616	10.0.0.1	10.0.0.2	TCP	66 5000 → 40912 [FIN, ACK] Seq=12 Ack=1049 Win=44544 Len=0 TSval=961093097 TS
40 1.472321198	10.0.0.1	10.0.0.2	TCP	74 5000 → 40920 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM Tsv

Figure 6: TCP connection close in persistent mode

7.2 New Connection Mode Screenshot



The screenshot shows a Wireshark packet capture on interface s1-eth1. The top bar indicates 'tcp.port == 5000'. The packet list on the left shows 96 packets. The packet details pane on the right shows the selected packet (No. 57) as a TCP Reset (RST) with Seq=12, Ack=1049, Win=44544, Len=0, and TSval=2842345788. The packet bytes pane shows the raw data. The status bar at the bottom indicates 'Packets: 96 - Displayed: 96 (100.0%)' and 'Profile: Default'.

No.	Time	Source	Destination	Protocol	Length	Info
57	2.283739075	10.0.0.1	10.0.0.2	TCP	66	5000 → 55638 [FIN, ACK] Seq=12 Ack=1049 Win=44544 Len=0 TSval=2842345788 TSecr=3205
58	2.443994445	10.0.0.2	10.0.0.1	TCP	66	55646 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=3794891167 TSecr=3205
59	2.444005236	10.0.0.2	10.0.0.1	TCP	66	55638 → 5000 [ACK] Seq=1049 Ack=13 Win=42496 Len=0 TSval=744082951 TSecr=2
60	2.444006278	10.0.0.2	10.0.0.1	TCP	1113	55646 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=42496 Len=1047 TSval=3794891167 TSecr=3
61	2.494159354	10.0.0.1	10.0.0.2	TCP	66	5000 → 55646 [ACK] Seq=1 Ack=1048 Win=44544 Len=0 TSval=3205801389 TSecr=3
62	2.494426727	10.0.0.1	10.0.0.2	TCP	77	5000 → 55646 [PSH, ACK] Seq=1 Ack=1048 Win=44544 Len=11 TSval=3205801389 TSecr=3
63	2.647208273	10.0.0.2	10.0.0.1	TCP	66	55646 → 5000 [ACK] Seq=1048 Ack=12 Win=42496 Len=0 TSval=3794891373 TSecr=3
64	2.647212023	10.0.0.2	10.0.0.1	TCP	66	55646 → 5000 [FIN, ACK] Seq=1048 Ack=12 Win=42496 Len=0 TSval=3794891373 TSecr=3
65	2.647212231	10.0.0.2	10.0.0.1	TCP	74	55656 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=35184705 TS
66	2.697264100	10.0.0.1	10.0.0.2	TCP	74	5000 → 55656 [ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TS
67	2.697261475	10.0.0.1	10.0.0.2	TCP	66	5000 → 55646 [FIN, ACK] Seq=12 Ack=1049 Win=44544 Len=0 TSval=3205801592 T
68	2.850781007	10.0.0.2	10.0.0.1	TCP	66	55656 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=3518470725 TSecr=3915
69	2.850785099	10.0.0.2	10.0.0.1	TCP	66	55646 → 5000 [ACK] Seq=1049 Ack=13 Win=42496 Len=0 TSval=3794891580 TSecr=3
70	2.850862348	10.0.0.2	10.0.0.1	TCP	1113	55656 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=42496 Len=1047 TSval=3518470725 TS
71	2.901061924	10.0.0.1	10.0.0.2	TCP	66	5000 → 55656 [ACK] Seq=1 Ack=1048 Win=44544 Len=0 TSval=3915356565 TSecr=3
72	2.901068424	10.0.0.1	10.0.0.2	TCP	77	5000 → 55656 [PSH, ACK] Seq=1 Ack=1048 Win=44544 Len=11 TSval=3915356565 T
73	3.051174613	10.0.0.2	10.0.0.1	TCP	66	55656 → 5000 [ACK] Seq=1048 Ack=12 Win=42496 Len=0 TSval=3518470925 TSecr=3
74	3.051226488	10.0.0.2	10.0.0.1	TCP	66	55656 → 5000 [FIN, ACK] Seq=1048 Ack=12 Win=42496 Len=0 TSval=3518470925 T
75	3.051252862	10.0.0.2	10.0.0.1	TCP	74	55668 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=38499264 TS
76	3.101307981	10.0.0.1	10.0.0.2	TCP	74	5000 → 55668 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TS
77	3.101316522	10.0.0.1	10.0.0.2	TCP	66	5000 → 55656 [FIN, ACK] Seq=12 Ack=1049 Win=44544 Len=0 TSval=3915356765 T
78	3.255513100	10.0.0.2	10.0.0.1	TCP	66	55668 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=3849926626 TSecr=1378
79	3.255526399	10.0.0.2	10.0.0.1	TCP	66	55656 → 5000 [ACK] Seq=1049 Ack=13 Win=42496 Len=0 TSval=3518471125 TSecr=3
80	3.255526767	10.0.0.2	10.0.0.1	TCP	1113	55668 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=42496 Len=1047 TSval=3849926626 TS
81	3.307588205	10.0.0.1	10.0.0.2	TCP	66	5000 → 55668 [ACK] Seq=1 Ack=1048 Win=44544 Len=0 TSval=1378902813 TSecr=3
82	3.307602872	10.0.0.1	10.0.0.2	TCP	77	5000 → 55668 [PSH, ACK] Seq=1 Ack=1048 Win=44544 Len=11 TSval=1378902814 T
83	3.466221878	10.0.0.2	10.0.0.1	TCP	66	55668 → 5000 [ACK] Seq=1048 Ack=12 Win=42496 Len=0 TSval=3849926837 TSecr=3
84	3.466226378	10.0.0.2	10.0.0.1	TCP	66	55668 → 5000 [FIN, ACK] Seq=1048 Ack=12 Win=42496 Len=0 TSval=3849926837 T
85	3.466226628	10.0.0.2	10.0.0.1	TCP	74	55674 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=126080004 TS
86	3.517348406	10.0.0.1	10.0.0.2	TCP	74	5000 → 55674 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TS
87	3.517356447	10.0.0.1	10.0.0.2	TCP	66	5000 → 55668 [FIN, ACK] Seq=12 Ack=1049 Win=44544 Len=0 TSval=1378903024 T
88	3.667617052	10.0.0.2	10.0.0.1	TCP	66	55674 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=1260800068 TSecr=3660
89	3.667620260	10.0.0.2	10.0.0.1	TCP	66	55668 → 5000 [ACK] Seq=1049 Ack=13 Win=42496 Len=0 TSval=3849927042 TSecr=3
90	3.668990709	10.0.0.2	10.0.0.1	TCP	1114	55674 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=42496 Len=1048 TSval=1260800090 TS
91	3.719843161	10.0.0.1	10.0.0.2	TCP	66	5000 → 55674 [ACK] Seq=1 Ack=1049 Win=44544 Len=0 TSval=3660743624 TSecr=3
92	3.719883161	10.0.0.1	10.0.0.2	TCP	78	5000 → 55674 [PSH, ACK] Seq=1 Ack=1049 Win=44544 Len=12 TSval=3660743624 T
93	3.873766985	10.0.0.2	10.0.0.1	TCP	66	55674 → 5000 [ACK] Seq=1049 Ack=13 Win=42496 Len=0 TSval=1260800090 T
94	3.873771002	10.0.0.2	10.0.0.1	TCP	66	55674 → 5000 [FIN, ACK] Seq=1049 Ack=13 Win=42496 Len=0 TSval=1260800090 T

Figure 7: Multiple TCP handshakes in new connection mode

8 Discussion

The persistent mode performed better because it reused a single TCP connection for all messages. This avoided repeated handshakes and reduced connection setup overhead. New connection mode showed higher response times and lower throughput because each message required a separate TCP connection.

As message size increased, throughput also increased. This happened because the fixed TCP overhead was spread across more data bytes for larger messages. Wireshark confirmed that persistent mode used one handshake and one close, while new connection mode produced repeated handshakes and closes.

9 Answers to Questions

Q1: Why does TCP use a three-way handshake?

TCP uses a three-way handshake to establish a reliable connection, synchronize sequence numbers, and confirm that both client and server are ready to communicate.

Q2: Which mode had lower response time?

Persistent mode had lower response time because the connection was established once and reused for multiple messages.

Q3: Why does new-connection mode have more overhead?

New-connection mode has more overhead because every message requires a new handshake and connection teardown, which adds extra packets and delay.

Q4: Which mode is better for CampusChat and why?

Persistent mode is better for CampusChat because chat applications send many messages over the same session, so keeping one connection open is more efficient.

Q5: Which mode is suitable for NetAttend and why?

Persistent mode is also suitable for NetAttend because attendance or form submissions may involve repeated exchanges with the same server session, which benefits from reduced connection overhead.

Q6: How does message size affect throughput?

Throughput increases with message size because larger messages carry more payload data for the same relative connection overhead.

Q7: How did Wireshark help verify TCP behavior?

Wireshark helped verify the presence of TCP handshakes, data transfer, and connection termination. It clearly showed the difference between persistent and new connection modes.

10 Conclusion

This experiment successfully demonstrated TCP communication in Mininet and compared persistent and new connection modes. The results showed that persistent mode is more efficient, with lower response time and higher throughput. Wireshark confirmed the TCP handshake and connection behavior as expected.